Spickzettel – Kurzpräsentation

Datenexfiltration über ICMP · Programm A/B

Vor dem Start

- Methode ist ICMP. Programm A liest `stgb_202c_202d.txt` (§202c/d StGB), Programm B empfängt, decodiert, zeigt an und speichert.
- Getrennte Prozesse, Kommunikation nur über den echten Netzwerkstack, kein direkter Funktionsaufruf.
- Zuerst B starten, dann A:

```
sudo .venv/bin/python programm_b.py
sudo .venv/bin/python programm_a.py
```

Während die Daten durchlaufen

- Codierung: Base64 – keine Verschlüsselung nötig, reicht laut Aufgabenstellung als Codierungsverfahren.
- Jedes Paket ist ein ICMP Echo-Request (Typ 8), 56 Byte Payload – wie ein normaler Ping.
- Eigenes 20-Byte-Rahmenformat in der Payload: Magic `NSEC`, Version, Pakettyp (INIT/DATA/FIN/ACK), Sequenznummer, Gesamtzahl, Datenlänge, CRC32 – danach 36 Byte codierte Daten.
- Ablauf: erst INIT (SHA-256 der Originaldatei), dann 61 DATA-Pakete, zum Schluss FIN.
- B antwortet auf jedes Paket mit Echo-Reply (Typ 0) – das ist zugleich die Bestätigung (ACK oder NACK).

Integrität & Fehlerkorrektur

- Pro Paket: CRC32 über den Datenteil, B prüft beim Empfang.
- Gesamtdatei: SHA-256 – A schickt den Hash im INIT, B vergleicht ihn am Ende.
- Fehlerkorrektur: Stop-and-Wait-ARQ. A wartet 2 s auf Bestätigung, bei Timeout/NACK bis zu 5 Wiederholungen.

Am Ende

- B zeigt den vollständigen Gesetzestext auf dem Bildschirm und speichert ihn in `output/empfangen_b.txt`.
- Beide Programme schreiben währenddessen ihre eigene pcap-Datei – ungefiltert, beide Richtungen.
- Nachweis bei Rückfrage: `diff stgb_202c_202d.txt output/empfangen_b.txt` muss leer sein.

Mögliche Rückfragen

- **Warum ICMP und nicht ARP/SNMP/DNS?** Firewalls lassen ICMP meist durch, Ping fällt nicht auf, Request/Reply liefert direkt einen Rückkanal für Bestätigungen.
- **Wie erkennt man das als Verteidiger?** Ungewöhnlich viele/regelmäßige Pings, Nutzlast mit hoher Entropie, IDS wie Snort/Suricata, Rate-Limiting + Egress-Filter.
- **Ist das sicher gegen Fälschung?** Nein – CRC32 ist keine Authentifizierung, nur Fehlererkennung. Jeder im Netz könnte theoretisch gültige Pakete fälschen.